

Infrastructure as Documentation:

Designing IaC That Explains Itself

DevOpsCon New York

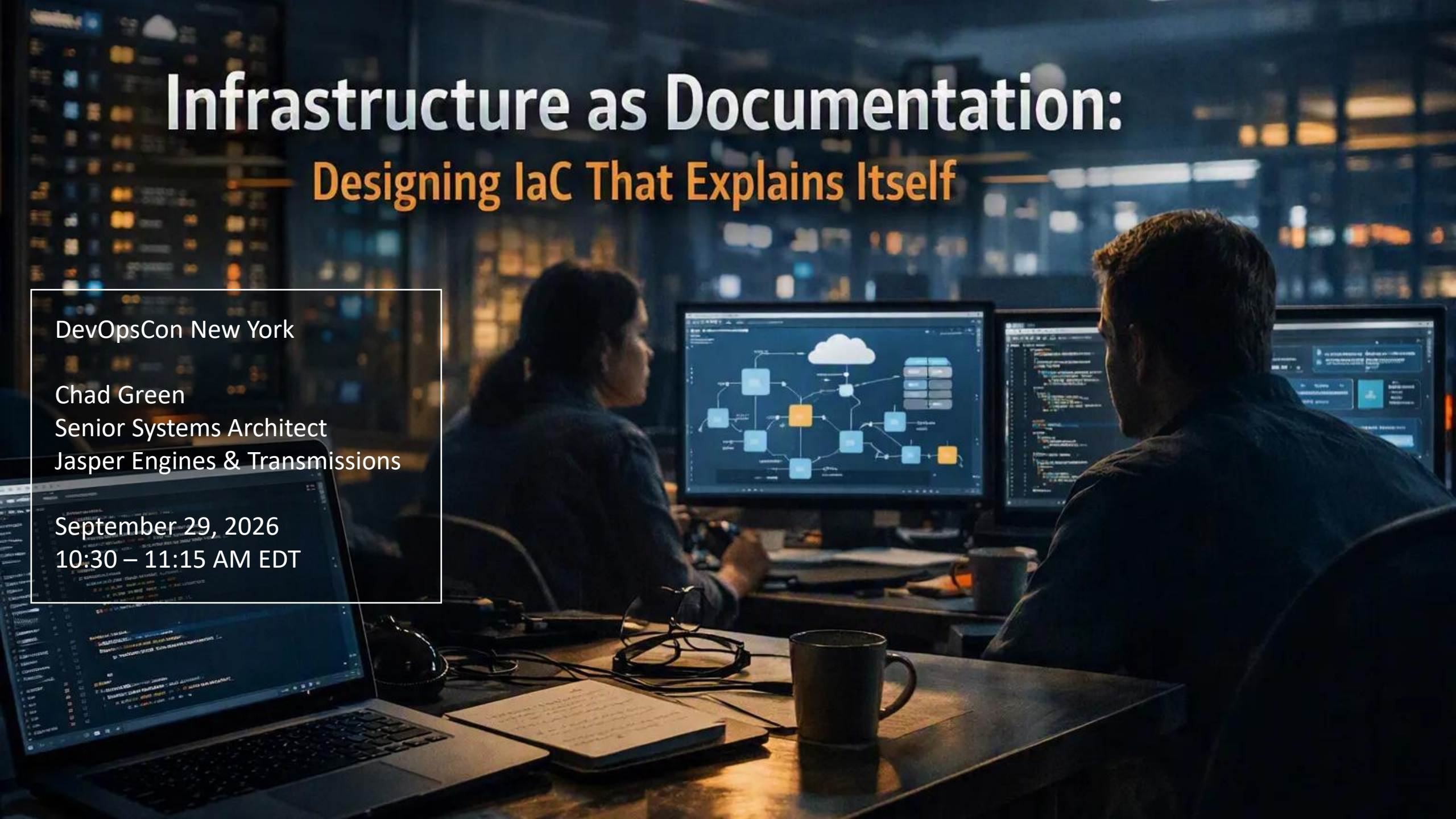
Chad Green

Senior Systems Architect

Jasper Engines & Transmissions

September 29, 2026

10:30 – 11:15 AM EDT





The Consequences

Fear to Change

New engineers were afraid to touch anything

Blind Reviews

PR reviews became rubber stamps; no one knew what they were approving

Governance Drift

Governance drifted into a Confluence page nobody read

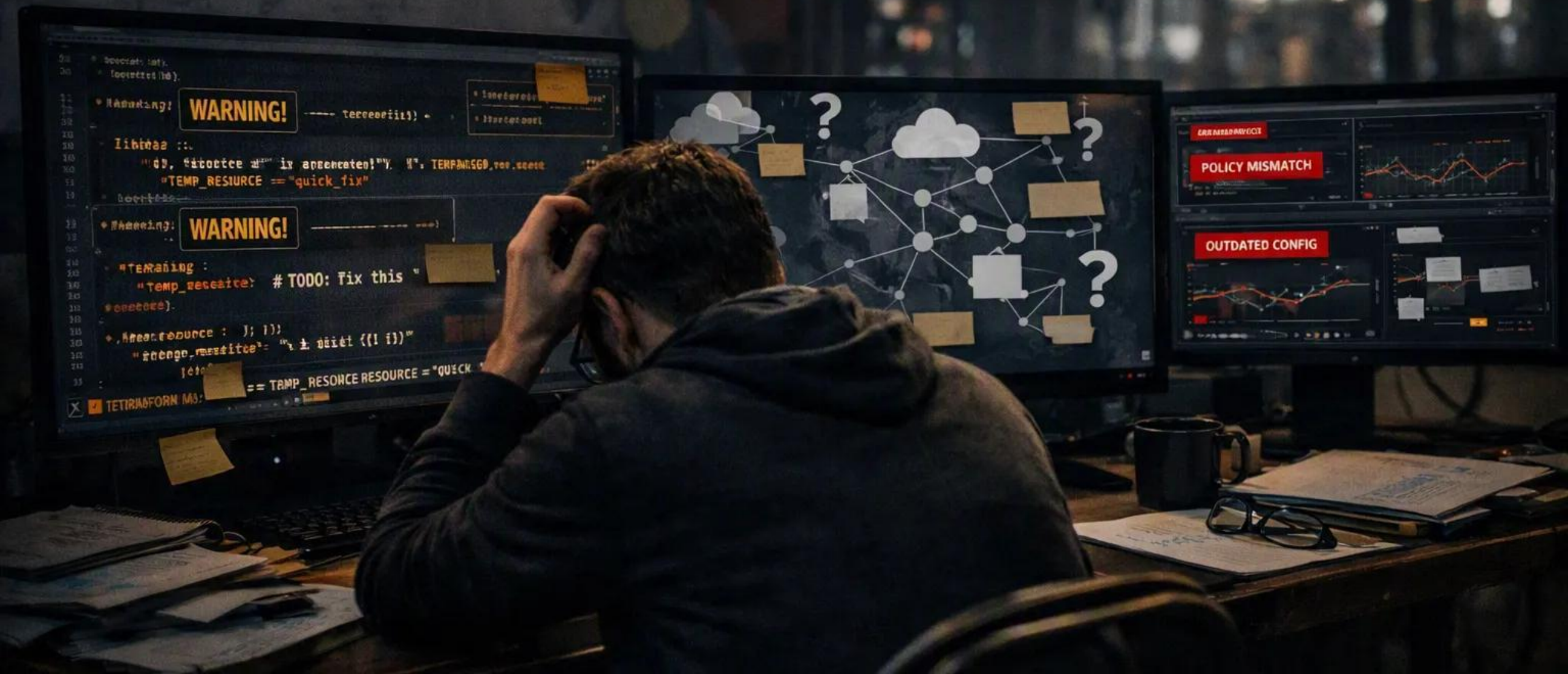
Haunted Deploys

Every deployment felt like spelunking in a haunted forest

Terraform wasn't the problem. *The intent had disappeared.*

The Problem: IaC Rot and Invisible Intent

Three failure modes. Every team has them.



What IaC Rots



Failure Mode 1

Invisible Intent

Code shows **what it builds**; never **why**. Names, variables, and outputs carry zero explanation of purpose or decision rationale.

👁️ The *why* behind every resource is invisible to future engineers.



Failure Mode 2

Governance Drift

Standards documents **diverge from the infrastructure they govern**. Policy lives in Confluence; the code never gets the memo.

📄 The PDF was perfect. The infrastructure was not.



Failure Mode 3

Review Fatigue

PRs become **rubber stamps** when context is missing. Reviewers can't catch what they can't understand, so they approve and hope.

👤 LGTM becomes the most dangerous phrase in infrastructure.



Show of hands: How many of you inherited Terraform you didn't write and didn't immediately understand?

Aurora's Reality



Aurora's Reality

</> The Code

```
main_vnet
```

```
// wasn't the main VNet
```

```
env
```

```
// no description, no validation
```

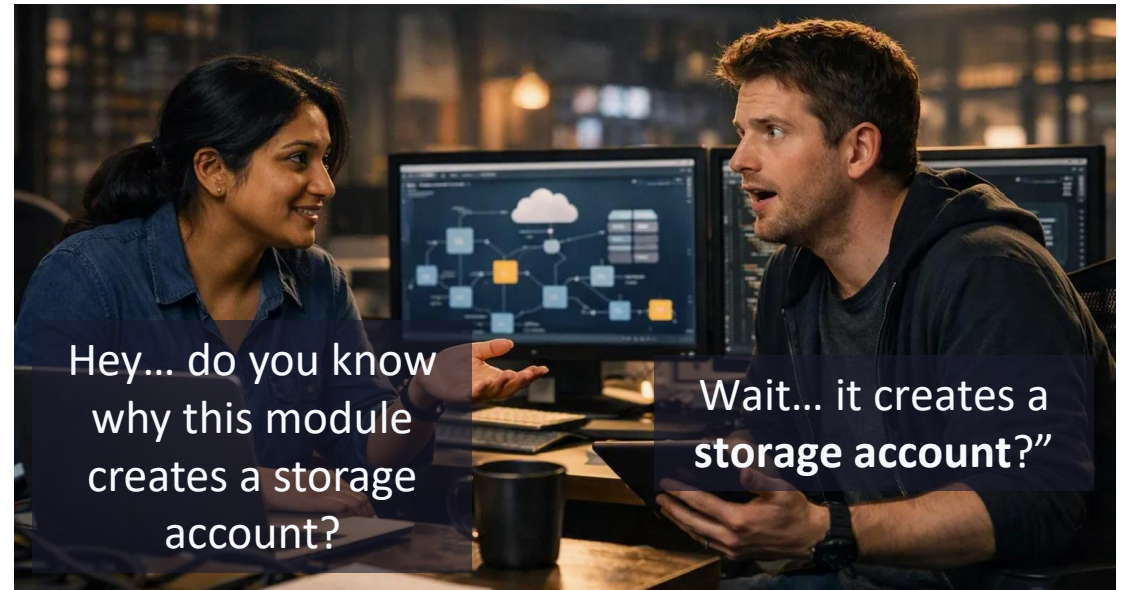
```
all_subnet_ids
```

```
// a flat list of implementation details
```

```
resource azurerm_storage_account "example" {  
  name                = "storageaccountname"  
  resource_group_name = azurerm_resource_group.this.name  
  location            = azurerm_resource_group.this.location  
  account_tier        = "Standard"  
  account_replication_type = "GRS"  
}
```

⚠ The Cost

- New engineers can't orient themselves
- Reviews can't catch what reviewers don't understand



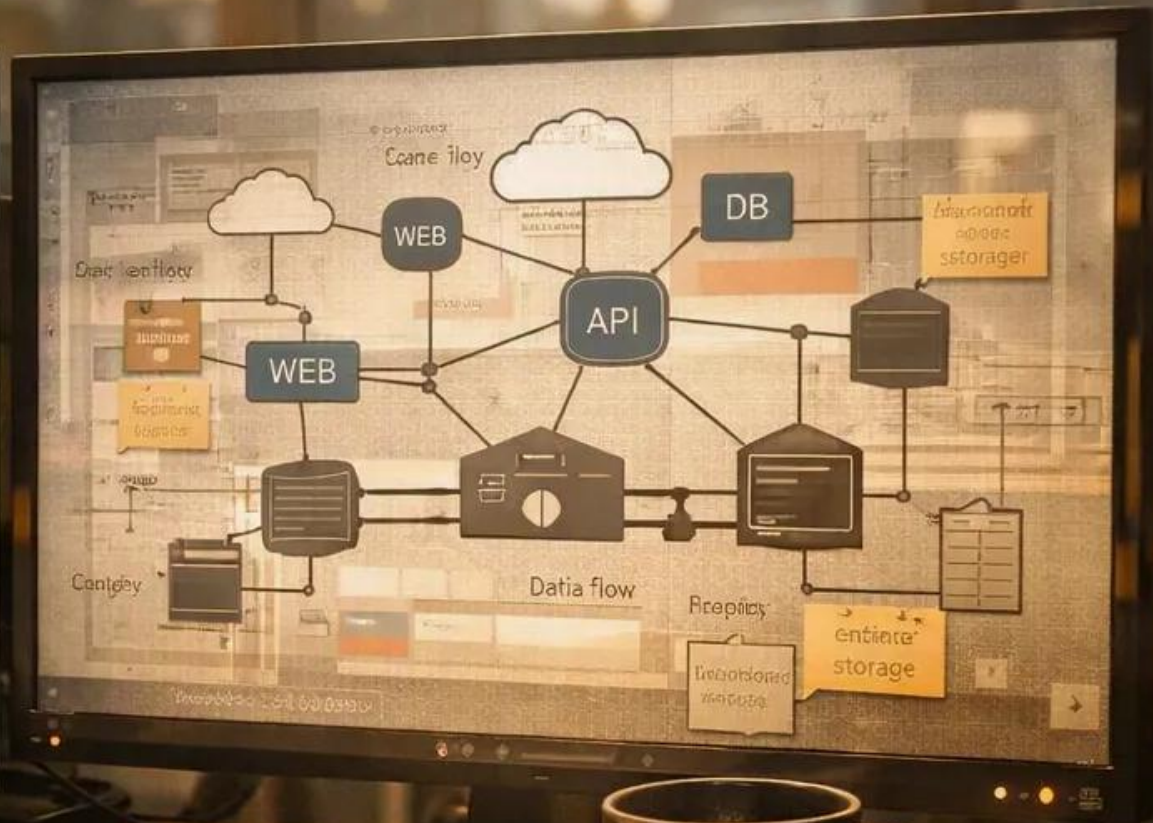
Hey... do you know
why this module
creates a storage
account?

Wait... it creates a
storage account?"

The Reframe: Infrastructure as Documentation

A different lens on what IaC is fundamentally for.

```
<module "vpc" app_network: {  
  description = "VPC for application:  
  cidr_block = "10.0.0.0/16"  
  
  <variable "project_env" { → Project Environment,  
    description = "Project Environment"  
    type = "string" = "production"  
  
  <output "vpc_id" {  
    < description = "VPC IP" → Value=module.vpc.id  
  < // Docs: Security group 'web_sg' allows HTTP& HTTPS traffic  
  
  resource "aws_security_group" web_sg {  
    < { name = "Web SG"  
    < { default = "protocol=*" < ingress
```



LSTN

"IaC is not automation — it's a communication artifact."

Written for people to read, reason about, review, and build on; that also happens to be executable.

“Modules are not scripts — they are contracts.”

A module defines an agreement between the producer and every future consumer: what it accepts, what it guarantees, and what it will never do implicitly.

“Variables and outputs are documentation surfaces.”

This is the first thing a new engineer reads when they open a module. Every name, description, default, and validation rule is a communication decision.

“Naming is architecture in language.”

Every resource name, every module name, every variable name is an architectural decision made visible (or invisible) by how deliberately you choose your words. The name is not decoration. The name *is* the documentation.

Infrastructure as Documentation

“IaC is not automation — it’s a communication artifact.”

“Modules are not scripts — they are contracts.”

“Variables and outputs are documentation surfaces.”

“Naming is architecture in language.”



The Contract-First Mindset

Four pillars for designing infrastructure that communicates intent.



Intent

What is this module for? One sentence before the first resource block.



Boundaries

Modules map to architectural boundaries, not convenience groupings.



Assumptions

Every assumption must be visible, not buried in defaults.



Contracts

Outputs declare stable, named, described interfaces; not implementation leaks.

The Contract-First Mindset

Four pillars for designing infrastructure that communicates intent.



Intent



Boundaries



Assumptions



Contracts



“Each pillar eliminates a category of invisible failure.”

The Pattern Library: How Code Explains Itself

Five patterns. Portable.

Grounded in contract-first thinking.

Intent-First Naming

```
bucket_name = "audit_logs"
```

Audit Logs

Assumption Surfacing

```
default_region = "us-east-1"
```

Default Setting

Outputs as Contracts

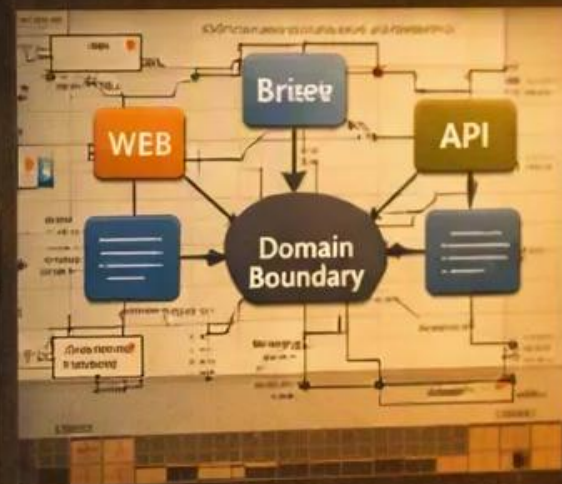
```
output "db_host" {  
  value = db.endpoint
```

Stable Interface

Opinionated Defaults

```
encryption_enabled = true
```

Team Standard:
Encryption On



Pattern Library Overview

Five portable patterns, each named, explained, and tied to contract-first principles. Works across every IaC tool.

1

Intent-First Naming

Names describe purpose, not implementation



2

Assumption Surfacing

Defaults encode visible, enforceable decisions



3

Boundary-Driven Module Structure

Modules map to architectural domains, not convenience groupings



4

Outputs as Contracts

Expose stable interfaces, not implementation details



5

Opinionated Defaults

Encode your team's standards, not the provider's factory settings



Pattern 1: Intent-First Naming

⚠ Naming That Lies

main_vnet

Was not the main VNet

db1

Which database? For which service?

sg-main

The main SG for what?

env

What values are valid?

✗ The Damage

- ✗ Communicates wrong intent — worse than no name at all
- ✗ New engineers guess; guesses calcify into accidental architecture
- ✗ Bad naming is a lie encoded in infrastructure



Pattern 4: Outputs as Contracts – The Leaky Version

- Output: `all_subnet_ids` is a flat list of every subnet ID in the module, **in order of creation**
- Three downstream modules accessed subnets by index position `[0]` app tier, `[1]` data tier, `[2]` management
- Team added a DMZ subnet → **indices shifted** → three modules broke silently at deployment, **Friday at 11 PM**



LEAKY OUTPUT

BEFORE DMZ SUBNET

```
[0] app tier
[1] data tier
[2] management
```

AFTER DMZ ADDED

```
[0] dmz ← shifted
[1] app tier ← shifted
[2] data tier ← shifted
[3] management ← shifted
```

3 modules broke silently

⚠ "That is not an output. That is a leak."

Patterns 4 & 5: Contracts + Opinionated Defaults

⚠️ Leaky Output

`all_subnet_ids`

Undifferentiated flat list — consumers depend on index

positions: `[0]` app tier

`[1]` data tier

`[2]` management

Add one subnet → indices shift → three modules break silently at 11 PM on a Friday.

✅ Contract-Driven Output

`app_tier_subnet_id`

`data_tier_subnet_id`

`management_subnet_id`

Singular, named, architectural role. Stable interface; adding subnets does not break consumers.

Pattern 5: Opinionated Defaults



Private endpoint enabled by default

Not public access



Deletion protection on by default



Minimum TLS version enforced

What your security policy requires — not the provider's factory setting

"The default is the policy."

Thank You

Chad Green

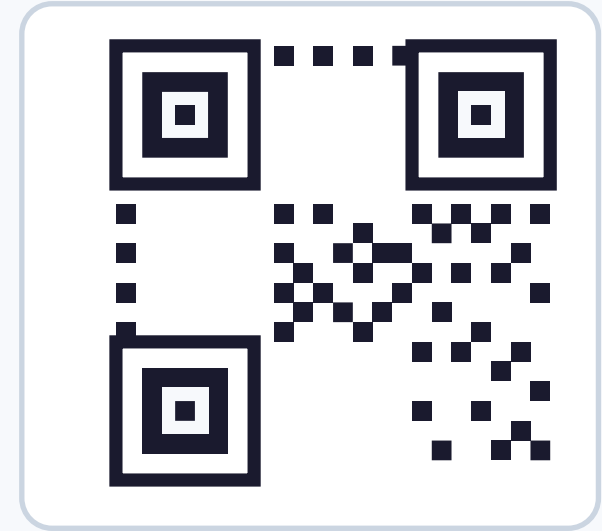
Senior Systems Architect
Jasper Engines & Transmissions

✉ chadgreen@chadgreen.com

🌐 chadgreen.com

🐦 @chadgreen

🌐 ChadwickEGreen



Portable Pattern Card + Slides

DevOpsCon New York

September 29, 2026 · 10:30 AM EDT

"If any part of Aurora's story sounded familiar — come find me."